

E.C.O.S.

Evolutionary Complex Organism Simulator

(Simulador de Organismos Complexos Evolutivos)

Luiza Mitsue Yano

SUMÁRIO

INTRODUÇÃO	1
COMO FUNCIONA A IA?	1
REDE NEURAL.....	1
A SIMULAÇÃO	4
OS ORGANISMOS	4
DEFINIÇÕES DE COMPORTAMENTO.....	7
ALIMENTAÇÃO	7
VALORES PADRÃO DO PROJETO	7
TECNOLOGIAS UTILIZADAS	8
FRONTEND	8
BACKEND	8
RENDERIZAÇÃO GRÁFICA	9
RESULTADOS	10
CONSIDERAÇÕES FINAIS	12

INTRODUÇÃO

O E.C.O.S. é uma simulação interativa 2D de seleção natural e autonomia biológica. A aplicação recria um ecossistema dinâmico onde organismos virtuais nascem, buscam recursos, consomem energia, reproduzem-se transmitindo características genéticas e morrem.

Cada indivíduo é guiado por um cérebro artificial baseado em uma pequena rede neural, responsável por definir seus movimentos a partir das leituras de seus sensores visuais. O ambiente divide-se entre herbívoros, que se alimentam de recursos gerados pelo mapa, e carnívoros, que caçam para sobreviverem. Aqueles que acumulam nutrição suficiente geram descendentes com pequenas variações genéticas, enquanto a exaustão metabólica ou o envelhecimento marcam o fim do seu ciclo de vida.

COMO FUNCIONA A IA?

Rede neural

`/frontend/src/simulation/NeuralNetwork.js`

Uma rede neural é responsável por receber informações do ambiente, processá-las e gerar uma decisão. Ela é instanciada da seguinte forma:

```
const brain = new NeuralNetwork(a, b, c)
```

Onde:

- a** → quantidade de neurônios de entrada (input nodes), que representam os sensores da criatura.
- b** → quantidade de neurônios da camada oculta (hidden nodes), responsáveis pelo processamento das informações recebidas.
- c** → quantidade de neurônios de saída (output nodes), responsáveis por gerar as decisões ou ações da criatura.

Após a criação da rede, são inicializados:

Pesos (weights): Conectam os neurônios entre as camadas e determinam a influência de cada entrada sobre o próximo neurônio. Quanto maior o peso, maior a influência daquela entrada na decisão da rede.

Biases (bias): Valores adicionais atribuídos a cada neurônio que servem como uma tendência inicial de ativação, garantindo que a rede possa reagir mesmo quando as entradas forem nulas.

Esses valores serão utilizados pela rede para transformar as entradas (sensores) em saídas (ações), sendo posteriormente ajustados durante o processo de aprendizado da inteligência artificial.

1. Método predict()

Esse método é responsável por realizar a indução (feedforward) da rede neural, ou seja, transformar os valores recebidos pelos sensores em decisões da criatura. O processo ocorre em duas etapas:

a. Processamento da camada oculta

Cada neurônio da camada oculta recebe todas as entradas da rede. O valor de cada entrada é multiplicado pelo peso correspondente. Em seguida, esses resultados são somados, adiciona-se o bias do neurônio e aplica-se a função de ativação $\tanh$ (tangente hiperbólica).

b. Processamento da camada de saída

Os valores calculados na camada oculta tornam-se as entradas da camada de saída. Novamente, realiza-se a soma dos pesos, adiciona-se o bias e aplica-se a função $\tanh$.

A tangente hiperbólica é uma função matemática não linear que comprime qualquer número real recebido para um intervalo fixo entre -1 e 1. Ela é fundamental no E.C.O.S. por dois motivos principais:

- ♦ Não linearidade - Permite que a rede aprenda relações complexas entre os sensores, e não apenas combinações simples.
- ♦ Normalização de saída - Garante que as decisões da criatura permaneçam dentro de limites físicos controlados.

Retorno:

O método retorna um vetor de dois números contido no intervalo definido, onde cada posição mapeia uma ação da criatura. Exemplo:

[-0.35, 0.82]

O primeiro valor representa a rotação e o segundo representa a intensidade da aceleração.

2. Método mutate()

Esse método é responsável por introduzir pequenas alterações na rede neural, simulando uma mutação genética. Em algoritmos evolutivos, as criaturas mais bem-sucedidas geram descendentes, mas se os filhos fossem cópias exatas dos pais, nenhuma característica nova surgiria. A mutação resolve isso alterando aleatoriamente alguns pesos e biases.

- a. Taxa de Mutação: Cada peso ou bias possui 8% de chance (rate = 0.08) de sofrer mutação.
- b. Processo: Quando a mutação ocorre, o valor original recebe um ajuste aleatório entre -0.5 e +0.5.
- c. Aplicação: A mutação é aplicada em todas as conexões da rede (pesos e biases de todas as camadas).

Com isso, cada nova geração herda a base da geração anterior, mas com pequenas diferenças que permitem a evolução do ecossistema.

3. Método clone()

Esse método cria uma cópia da rede neural, ou seja, uma nova instância da classe `NeuralNetwork` é gerada com a mesma quantidade de neurônios de entrada, camada oculta e saída, e todos os pesos e biases da rede original são copiados para essa nova instância.

Durante a cópia, são criados novos arrays e matrizes, garantindo que a rede clonada seja independente da original. Alterações feitas na cópia (como mutações) não afetam a rede original.

Esse mecanismo é usado durante a reprodução, permitindo que os descendentes herdem a estrutura do cérebro dos pais antes da mutação.

4. Métodos Auxiliares

Responsáveis por inicializar pesos e biases com valores aleatórios, garantindo que cada nova instância comece com uma configuração diferente.

`createRandomArray()`: Cria um vetor (lista) de tamanho definido, preenchido com números aleatórios entre -1 e 1. É utilizado para inicializar os biases de uma camada e serve como base para a criação das matrizes.

`createRandomMatrix()`: Cria uma matriz (tabela) preenchida com valores aleatórios entre -1 e 1, gerando cada uma de suas linhas através da função `createRandomArray()`. É utilizada para inicializar os pesos de todas as conexões entre duas camadas de neurônios.

A SIMULAÇÃO

Ter um cérebro capaz de tomar decisões não é suficiente se a criatura não tiver um corpo para executá-las nem um mundo onde agir. Para que a inteligência artificial ganhe vida, precisamos dar um conjunto de traços corporais, noções de posição, limites biológicos e um ecossistema com regras bem definidas.

As próximas seções serão dedicadas à abordagem da física e do comportamento real das entidades, detalhando seus atributos físicos e a forma como interagem com o ambiente.

Os organismos

📁 [frontend/src/simulation/Creature.js](#)

A classe `Creature` representa cada organismo do ecossistema, seja herbívoro ou carnívoro. Cada instância tem posição e física simples de movimento, um cérebro, e um conjunto de regras biológicas de energia, idade, reprodução e morte que variam de acordo com a espécie e com a configuração ativa no momento.

Ela é instanciada da seguinte forma:

```
const creature = new Creature(x, y, brain, type, traits)
```

Onde:

x, y → posição inicial no canvas.

brain → rede neural.

type → dieta.

traits → Define se usará os atributos padrão do sistema ou se herdará atributos de um pai. (velocidade, tamanho do sensor, energia, etc.).

Além da posição e do ângulo inicial, o construtor define os atributos de controle e o ciclo de vida da criatura. Entre eles estão a linhagem à qual o organismo pertence, a idade, a energia inicial e máxima, e o estado do indivíduo (vivo ou morto).

Também é determinado o tempo de vida máximo da criatura, sorteado em um intervalo entre 1800 e 5400 frames, o que corresponde a uma longevidade de 30 a 90 segundos em uma simulação executada a 60 quadros por segundo.

1. Método *update()*

Executado a cada frame, este método é o núcleo do comportamento e do ciclo vital do organismo.

```
update(foodList, creatureList, newCreatures, canvasWidth, canvasHeight)
```

Ele recebe o estado do mundo e executa a seguinte sequência:

- a. Envelhecimento: Interrompe a execução se a criatura estiver morta; caso contrário, incrementa sua idade.
- b. Alimentação: Processa a colisão com alimentos (para herbívoros) ou a caça (para carnívoros).
- c. Percepção e Decisão: Se detectar algum estímulo, consulta a rede neural para decidir a direção do giro e a aceleração. Se nada for visto, ativa o modo exploração para evitar que a criatura fique estática.
- d. Física e Movimento: Aplica a rotação ao ângulo de visão e ajusta a velocidade linear. Em seguida, calcula e atualiza as novas coordenadas x e y.
- e. Limites do Mapa: Ao atingir as bordas do canvas, trava a posição, zera a velocidade e sorteia um novo ângulo para reorientar o organismo
- f. Gasto Metabólico: Deduz a energia consumida no frame, somando quatro fatores:

$$\text{Gasto} = 0,02 + (\text{velocidade} * 0,03) + (|\text{giro}| * 0,05) + (\text{idade} * 0,000002)$$

2. Método *readSensors()*

Este método simula o sistema visual da criatura, retornando um vetor de 8 posições que é dividido em dois grupos de 4 direções, sendo elas: frente, direita, trás e esquerda.

- a. Sensores de comida ou presa (posições 0 a 3):

Para herbívoros, indicam a direção do item de comida mais próximo dentro do alcance; Para carnívoros, indicam a direção do herbívoro vivo mais próximo.

- b. Sensores de perigo (posições 4 a 7):

Para herbívoros, indicam a direção do carnívoro vivo mais próximo. Funcionando como um radar de predador para a rede neural aprender a fugir. Para carnívoros, permanecem sempre zerados, pois não têm predadores no ecossistema.

Cada valor é normalizado entre 0 (nada, ou fora de alcance) e 1 (bem próximo). O cálculo de qual dos 4 quadrantes recebe o valor de proximidade é feito por um método auxiliar compartilhado, *fillDirectionalSensors()*, reaproveitado tanto para o sensor de comida e presa quanto para o de perigo.

3. Método draw()

Desenha a criatura no canvas em sua posição e ângulo de visão atuais, utilizando transformações de renderização (translate e rotate). O corpo é representado como uma elipse com um pequeno olho indicando sua orientação. A cor base do organismo varia de acordo com sua dieta, verde-azulado (#368270) para herbívoros e dourado (#9b7924) para carnívoros.

Além da cor principal, o método aplica duas pistas visuais dinâmicas:

- a. **Aura de reprodução:** quando a energia da criatura está a menos de 20 pontos do seu limiar de reprodução, um brilho amarelo (#fff700) é aplicado ao corpo, sinalizando visualmente que ela está prestes a gerar descendentes.
- b. **Escurecimento por idade:** Uma camada preta transparente é desenhada sobre a criatura, ficando cada vez mais escura conforme ela envelhece (chegando a 80% de escurecimento no fim da vida). Isso faz com que indivíduos mais velhos fiquem visivelmente escurecidos em relação aos recém-nascidos.

4. Método reproduce()

Quando a energia acumulada atinge o seu limite, a criatura se reproduz de forma assexuada:

- a. **Energia:** Metade da energia da criatura-mãe é transferida para o filho, sendo descontada de seu corpo.
- b. **Herança genética:** O cérebro é copiado com e sofre uma mutação de até 8%.
- c. **Posição:** O filho nasce ao lado da mãe, com um deslocamento aleatório de até 15 pixels em cada eixo.
- d. **Geração:** O contador de linhagem do filho é incrementado em relação ao da mãe.

O novo indivíduo é adicionado temporariamente ao array `newCreatures`. Ao final do frame, a simulação transfere todos os filhotes dessa lista de espera para a população principal.

5. Métodos de Colisão

`checkFoodCollision()`: Usado apenas por herbívoros. Calcula a distância da criatura até a comida. Ao encostar no alimento, absorve sua energia, respeitando o limite máximo do corpo, e o item é removido do mapa.

`checkHuntingCollision()`: Usado apenas por carnívoros. Procura por herbívoros vivos no alcance de caça. Ao entrar em contato direto com uma presa, o carnívoro ganha 70 pontos de energia, respeitando seu limite máximo, e o herbívoro morre instantaneamente, sendo removido da simulação.

Definições de Comportamento

📁 [frontend/src/simulation/simulationConfig.js](#)

Este arquivo centraliza todos os parâmetros ajustáveis do ecossistema. Quando o usuário altera um valor no painel com a simulação rodando, o comportamento varia em dois grupos:

- a. **Em tempo real:** Taxa de surgimento (spawn) de comida e limites de superpopulação (comida, herbívoros e carnívoros).
- b. **Ao reiniciar:** População inicial, quantidade inicial de comida e velocidade das criaturas.

O método `resetSimulationConfig()` restaura todas as opções para os valores padrão do projeto. Para manter a sincronização, o algoritmo atualiza as configurações na memória existente em vez de criar um objeto novo, eliminando a necessidade de recarregar a aplicação.

Alimentação

📁 [frontend/src/simulation/Food.js](#)

A classe `Food` representa os alimentos disponíveis no mapa para os herbívoros. Ela armazena apenas a sua posição e o valor energético fornecido de 30 pontos de energia por padrão. Visualmente, é representada por um pequeno círculo roxo (#9948c1). O aparecimento de novos recursos no mapa é controlado em tempo real em segundos e não pela taxa de quadros. Isso garante que a taxa de surgimento de comida permaneça constante e idêntica, independentemente do desempenho ou dispositivo do usuário.

Valores padrão do projeto

Os valores exibidos no painel representam a configuração inicial recomendada para um ecossistema equilibrado. Eles foram calculados com base nas fórmulas de metabolismo da criatura, garantindo que os organismos consigam se reproduzir com facilidade no início da simulação.

- ♦ População Inicial: 30 herbívoros e 8 carnívoros.
- ♦ Limites Máximos: 120 herbívoros, 35 carnívoros e 290 alimentos.
- ♦ Oferta de Comida: 190 itens ao iniciar, com reposição contínua de 4 novos itens por segundo.
- ♦ Herbívoros: Velocidade 2.6, alcance visual de 150px e reprodução ao atingir 145 pontos de energia.
- ♦ Carnívoros: Velocidade 3.0, alcance visual de 150px e reprodução ao atingir 185 pontos de energia.
- ♦ Capacidade Energética: Todos os indivíduos nascem com 110 pontos de energia e podem acumular até o limite máximo de 200.

TECNOLOGIAS UTILIZADAS

A arquitetura do projeto foi estruturada para separar claramente a interface do usuário, a lógica da simulação e o modelo de inteligência artificial. Esta seção aborda uma visão geral do papel de cada tecnologia utilizada e da responsabilidade dos arquivos principais da aplicação.

Frontend

1. *React*

Biblioteca responsável por como a tela se parece e como o usuário interage com o site. Todo o painel de configurações, estatísticas e legenda são componentes React.

2. *Vite*

Liga um servidor local no computador e mostra as mudanças na tela na mesma hora em que o código é salvo, por padrão em `http://localhost:5173`, sem precisar recarregar a página manualmente.

3. *Canvas*

Elemento HTML5 usado para desenhar a simulação. Diferente do DOM tradicional, onde cada elemento fica guardado na memória do navegador, o Canvas funciona em modo imediato. Ele apenas recebe as instruções de desenho e as descarta em seguida, permitindo renderizar centenas de criaturas e itens de comida simultaneamente sem travar a tela.

Backend

1. *Node.js*

Ambiente que executa JavaScript fora do navegador, usado para rodar o servidor.

2. *Express*

Framework que simplifica a criação de rotas HTTP no Node.js. Atualmente o backend do projeto é mínimo e conta apenas com uma rota de teste (`GET /api/status`), servindo de base para futuras expansões, já que toda a simulação é processada diretamente no frontend.

Renderização Gráfica

1. *App.jsx*

Componente principal da aplicação React. É responsável por estruturar a tela, organizando o título, a barra de estatísticas, o painel de configurações, a área da simulação e a legenda do sistema.

2. *SimulationCanvas.jsx*

Desenha a simulação na tela e roda o ciclo contínuo do ecossistema a cada quadro. O espaço da simulação tem um tamanho fixo de 1300x800 pixels para calcular os movimentos das criaturas, mas a tela se adapta automaticamente ao tamanho do seu monitor ou celular sem esticar ou distorcer a imagem.

3. *Infopanel.js*

Exibe a legenda explicativa do ecossistema, detalhando o significado das cores, do brilho de reprodução e do escurecimento das criaturas com o passar da idade.

4. *Statspanel.js*

Barra informativa exibida abaixo do título que mostra, em tempo real, a contagem de herbívoros, carnívoros, comida disponível, a geração mais avançada e o tempo decorrido.

5. *Configpanel.js*

Painel lateral que permite ao usuário ajustar as regras do ecossistema sem mexer no código. As opções são organizadas por categorias (população inicial, limites de superpopulação, comida e comportamento), indicando se a alteração entra em vigor em tempo real ou se exige o reinício da simulação. Também conta com um botão para restaurar todos os parâmetros para os valores originais do projeto.

RESULTADOS

Esta seção apresenta os dados e observações coletados a partir de múltiplos cenários de teste realizados na aplicação. O objetivo é avaliar a estabilidade do ecossistema, a eficiência do modelo de inteligência artificial ao longo das gerações e o impacto das variações de parâmetros na dinâmica populacional de herbívoros e carnívoros.

O ciclo de colapso: mais herbívoros, mais comida

Em cenários com alta população inicial de herbívoros e fartura de comida, a simulação costuma seguir um padrão repetitivo de colapso:

1. Explosão Populacional: A abundância de alimento gera um crescimento rápido dos herbívoros, o que faz a comida disponível diminuir.
2. Superpopulação de Predadores: O aumento de presas favorece os carnívoros, que se multiplicam aceleradamente.
3. Efeito Dominó e Extinção: A caça excessiva zera a população de herbívoros; sem presas para se alimentar, os carnívoros também entram em colapso, levando à extinção total do ecossistema.

O equilíbrio perfeito é raro

Muitas vezes se imagina que a simulação ideal seria aquela com um ecossistema perfeitamente equilibrado entre as duas espécies. Contudo, na prática, os testes mostram como é difícil manter uma convivência estável de longo prazo. Quase sempre uma das espécies acaba extinta.

Quando os carnívoros extinguem os herbívoros eles acabam morrendo logo em seguida, pois sua única fonte de energia são as presas, diferente da comida vegetal, que surge automaticamente no mapa. Por conta dessa dependência, a extinção mais comum é a dos próprios carnívoros, permitindo que os herbívoros dominem o ecossistema nas gerações mais avançadas.

A evolução do comportamento

O aprendizado por seleção natural fica evidente ao acompanhar o avanço das gerações. Como iniciam com redes neurais totalmente aleatórias, as criaturas passam os primeiros instantes girando no próprio eixo, sem sair do lugar. No entanto, após poucos minutos e algumas gerações de mutação e seleção natural, elas já se deslocam de forma independente pelo mapa e passam a utilizar seus sensores para fugir ativamente de predadores.

Sorte Genética: Padrões Ineficientes Espalhados por Acaso

Também ocorre de comportamentos ineficientes (como andar em círculos, seguir em linha reta ou repetir trajetos curtos em vez de explorar o mapa) serem transmitidos para as gerações seguintes. Isso acontece quando, por sorte, esse movimento aleatório coloca a criatura perto de alimento ou de uma presa no momento exato, permitindo que ela se reproduza sem ter desenvolvido uma estratégia realmente útil.

Na maioria das vezes, essas linhagens acabam extintas. Como dependem exclusivamente da sorte para reencontrar recursos e as condições do mapa mudam a cada instante, é apenas uma questão de tempo até que o padrão pare de funcionar e a linhagem desapareça.

Assimetria estrutural entre as espécies

A estrutura do projeto e de suas configurações padrão cria uma assimetria natural entre as espécies.

Por um lado, os herbívoros são beneficiados por um recurso gerado automaticamente pelo ambiente. Já os carnívoros não possuem um equivalente e precisam aprender a caçar de forma eficiente para sobreviver.

Por outro lado, os herbívoros enfrentam uma complexidade comportamental maior, já que precisam aprimorar duas habilidades ao mesmo tempo, precisando buscar recursos e fugir de predadores. Os carnívoros, por estarem no topo da cadeia alimentar, não sofrem pressão de fuga e podem concentrar seu aprendizado exclusivamente na caça.

No fim, cada espécie lida com um desafio diferente, e a vantagem de uma sobre a outra na simulação depende de como essas pressões se equilibram a cada execução.

Padrões recorrentes ao final das simulações

Após a execução de múltiplos testes, foi possível identificar três cenários principais de desfecho para o ecossistema.

1. Extinção total das espécies: Quando a cadeia alimentar entra em colapso ou os recursos ficam escassos demais para sustentar a vida.
2. Hiperlotação de uma ou mais espécies: Quando a população cresce de forma descontrolada até atingir os limites do ecossistema.
3. Estagnação de gerações: Momento em que a população atinge um ponto ideal de consumo de energia e reprodução, estabilizando-se e parando de evoluir de forma perceptível.

Geração mais alta observada

Com as configurações padrão usadas no momento do teste, observou-se o seguinte resultado.

- ♦ Em 15 minutos de simulação, foram alcançadas 41 gerações de herbívoros.

Este é um valor registrado em uma execução específica e não deve ser visto como uma regra. O resultado varia a cada teste por causa da aleatoriedade da simulação, como mutações genéticas, posições iniciais e a ordem dos acontecimentos, principalmente em cenários iniciais mais favoráveis ou desfavoráveis.

CONSIDERAÇÕES FINAIS

No fim, os testes deixam claro que o E.C.O.S. não busca simular um ecossistema perfeitamente equilibrado, mas sim criar um ambiente simples o suficiente para observar como pressões de seleção natural moldam o comportamento de uma população ao longo de gerações. O resultado de cada execução é imprevisível por natureza.

Possíveis implementações futuras

- a. Tendência aleatória ou herdada dos pais para que as criaturas passem a andar em bando.
- b. Opção para exportar e salvar o histórico de uma simulação, facilitando a comparação de populações ao longo do tempo em diferentes teste .
- c. Painel de controle para ajustar atributos como alcance de sensor e limite de reprodução, que hoje ficam fixos no código.
- d. Um modo de inspeção para destacar visualmente a criatura da geração mais avançada e permitir que o usuário clique em qualquer indivíduo para visualizar seus atributos detalhados.

Com o passar do tempo, torna-se fascinante observar a dinâmica do ecossistema, acompanhando o ciclo contínuo de nascimento, crescimento, reprodução e morte dessas criaturas acontecendo em tempo real.